MedFlow: Comprehensive Line-by-Line Guide

File: `src/main/java/com/medflow/simulation/scheduler/SchedulerService.java` (586 Lines)
Purpose: CPU Process Scheduling Algorithms (FCFS, Priority Triage, Round Robin, SRTF) adapted for Hospital Emergency Operations.

🎓 **Master Viva Hook (Say this in your exam):**
"Sir/Ma'am, in a hospital, specialized resources like surgeons and operation theaters are strictly finite, exactly like CPU cores. Treatments are processes characterized by an Arrival Time (AT), Burst Time (BT), and Triage Priority. `SchedulerService.java` is our core operating system engine that schedules treatments and objectively measures average Waiting Time (WT) and Turnaround Time (TAT)."

📐 **Universal Scheduling Mathematical Formulas:**
1. **Completion Time (CT):** Timestamp when the process completely finishes execution.
2. **Turnaround Time (TAT):** Total duration from arrival to completion: `TAT = CT - AT`
3. **Waiting Time (WT):** Pure idle duration spent in ready queue: `WT = TAT - BT`
4. **Average Metrics:** `Avg WT = Σ(WT) / N`, `Avg TAT = Σ(TAT) / N`

Section 1: Package, Imports & Class Header (Lines 1 – 41)

Sets up the Spring Boot service container, DTO dependencies, and core scheduling documentation.

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
1	`package com.medflow.simulation.scheduler;`	Declares the Java package. Groups this service under the simulation module, keeping OS scheduling cleanly isolated from DB controllers.

3–6	`import com.medflow.dto.GanttBlockDto;`
	`import com.medflow.dto.ProcessTaskDto;`
	`import com.medflow.dto.ScheduleResponseDto;`
	`import org.springframework.stereotype.Service;`
		Imports:
		• `GanttBlockDto`: Models a single time-slice block on the Gantt chart timeline.
		• `ProcessTaskDto`: Models incoming patient processes (AT, BT, Priority).
		• `ScheduleResponseDto`: Wrapper returning the computed Gantt list, Avg WT, and Avg TAT.
		• `@Service`: Spring stereotype marking this class as an injectable business logic service.

8	`import java.util.*;`	Imports Java collections: `List`, `ArrayList`, `Queue`, `LinkedList`, `Comparator`.
10–39	`/** Javadoc documentation ... */`	Academic class-level documentation defining the OS-to-Hospital mapping and mathematical formulas for examiners.
40–41	`@Service` `public class SchedulerService {`	`@Service` tells the Spring IoC container to instantiate a singleton instance at startup and inject it into `SimulationController`.

Section 2: First-Come, First-Served (FCFS) (Lines 43 – 105)

Non-Preemptive FIFO Queue Routine walk-in consultations.

```
FCFS Gantt: [0--T1 (45m)--45] [45--T2 (30m)--75] [75--T3 (20m)--95] [95--T4 (50m)--145]
Avg WT: 41.25 min | Avg TAT: 77.50 min
```

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
54	`public ScheduleResponseDto scheduleFCFS(List<ProcessTaskDto> tasks) {`	Entry point for FCFS algorithm. Accepts list of patient tasks and returns the complete schedule response.
56	`List<ProcessTaskDto> processList = prepareTasks(tasks);`	Calls defensive helper `prepareTasks()`. If input is null/empty, loads canonical default hospital tasks from `oracle_seed.sql`.
59	`processList.sort(Comparator.comparingInt(ProcessTaskDto::getArrivalTime));`	**The Core FCFS Rule:** Sorts all processes strictly by Arrival Time ascending. Whichever patient arrives first is scheduled first.

61–64	`List<GanttBlockDto> ganttChart = new ArrayList<>();`
	`int currentTime = 0;`
	`double totalWaitingTime = 0;`
	`double totalTurnaroundTime = 0;`
		Initializes the timeline chart, simulation clock (`currentTime = 0`), and accumulators for statistical averages.

66	`for (ProcessTaskDto task : processList) {`	Loops sequentially through the arrival-sorted processes.

68–76	`if (currentTime < task.getArrivalTime()) {`
	`  ganttChart.add(new GanttBlockDto(`
	`  "IDLE",`
	`  "Idle (Awaiting Patient)",`
	`  currentTime,`
	`  task.getArrivalTime()`
	`  ));`
	`  currentTime = task.getArrivalTime();`
	`}`
		CPU IDLE State Handling: If the doctor finishes current work at minute 10, but the next patient does not arrive until minute 15, the doctor is IDLE for 5 minutes. Fast-forwards clock to `task.getArrivalTime()` and logs an IDLE block.

Line(s)	Java Source Code	Detailed Line-by-Line Explanation

78–81	`int startTime = currentTime;`
	`int endTime = startTime + task.getBurstTime();`
	`currentTime = endTime;`
		Non-Preemptive Execution: The process runs continuously from `startTime` until `endTime = startTime + BT`. The clock advances to `endTime`.

84–86	`int completionTime = endTime;`
	`int turnaroundTime = completionTime - task.getArrivalTime();`
	`int waitingTime = turnaroundTime - task.getBurstTime();`
		Standard OS Metric Calculations:
		• `CT = endTime`
		• `TAT = CT - AT`
		• `WT = TAT - BT`

87–88	`totalWaitingTime += waitingTime;` `totalTurnaroundTime += turnaroundTime;`	Adds the individual process's WT and TAT to the running totals for later average calculation.
90–92	`String label = task.getProcessId() + ": " + task.getPatientName() + " (" + task.getBurstTime() + "m)";` `ganttChart.add(new GanttBlockDto(task.getProcessId(), label, startTime, endTime));`	Creates a Gantt block for the UI containing process ID, patient name, procedure duration, start, and end times.

94–96	`int n = processList.size();`
	`double avgWait = roundTwoDecimals(totalWaitingTime / n);`
	`double avgTat = roundTwoDecimals(totalTurnaroundTime / n);`
		Computes average WT and TAT by dividing total sums by total process count n, rounded to 2 decimal places.

98–104	`return new ScheduleResponseDto(`
	`"FCFS (First-Come, First-Served)",`
	`ganttChart,`
	`avgWait,`
	`avgTat,`
	`"FCFS completed successfully..."`
	`);`
		Packages results into response DTO for the REST controller.

Section 3: Priority Scheduling & Emergency Preemption (Lines 107 – 265)

Preemptive & Non-Preemptive | Emergency Triage | Emergency (P1) preempts Routine (P3).

```
Preemptive Priority Gantt: [0--T1 (P1)--45] [45--T4 (P1)--95] [95--T2 (P2)--125] [125--T3 (P3)--145]
Avg WT: 53.75 min | Avg TAT: 90.00 min
```

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
111–113	`public ScheduleResponseDto schedulePriority(List<ProcessTaskDto> tasks) { return schedulePriority(tasks, true); }`	Convenience overload. Defaults to `preemptive = true` (emergency preemption triage mode).
126	`public ScheduleResponseDto schedulePriority(List<ProcessTaskDto> tasks, boolean preemptive) {`	Configurable priority scheduling method. Supports both preemptive and non-preemptive modes.
131–142	`class TaskState { final ProcessTaskDto original; final int index; int remainingBurst; int completionTime; TaskState(ProcessTaskDto original, int index) { this.original = original; this.index = index; this.remainingBurst = original.getBurstTime(); } }`	**Simulated Process Control Block (PCB):** Internal class modeling the OS kernel's PCB. Tracks dynamic state without altering original database models: • `remainingBurst`: Time units still needed on CPU. • `completionTime`: Recorded when remainingBurst reaches 0.
144–147	`List<TaskState> states = new ArrayList<>(); for (int i = 0; i < n; i++) { states.add(new TaskState(processList.get(i), i)); }`	Initializes a PCB state wrapper for every patient treatment in the batch.
149–154	`int completed = 0; int currentTime = states.stream().mapToInt(s -> s.original.getArrivalTime()).min().orElse(0); TaskState currentRunning = null; int blockStartTime = currentTime;`	Initializes simulation loop variables. `currentTime` starts at earliest patient arrival time. `completed` tracks how many tasks have finished.
156	`while (completed < n) {`	Main simulation loop. Continues until every single process has finished ($BT_{rem} = 0$).

Line(s)	Java Source Code	Detailed Line-by-Line Explanation

158–163	`List<TaskState> available = new ArrayList<>();`
	`for (TaskState s : states) {`
	`  if (s.original.getArrivalTime() <= currentTime && s.remainingBurst > 0) {`
	`  available.add(s);`
	`  }`
	`}`
		Ready Queue Filter: Collects all processes that have already arrived at or before `currentTime` and still require CPU time.

165–181	`if (available.isEmpty()) {`
	`  ... IDLE block logic ...`
	`  currentTime = nextArrival;`
	`  continue;`
	`}`
		If no process has arrived yet, flushes any currently running slice, records an IDLE block, jumps clock to next arrival, and continues loop.

185–189	`TaskState highest = available.stream()`
	`.min(Comparator.comparingInt((TaskState s) -> s.original.getPriority())`
	`  .thenComparingInt(s -> s.original.getArrivalTime())`
	`  .thenComparingInt(s -> s.index))`
	`  .orElse(available.get(0));`
		Highest Priority Selection:
		• Primary Sort: Lowest numerical priority value (1 = Emergency < 2 < 3).
		• Tie-Breaker 1: Earlier Arrival Time (AT).
		• Tie-Breaker 2: Original list insertion index.

191–205	`if (!preemptive) {`
	`  ... runs currentRunning to 0 ...`
	`  currentTime += currentRunning.remainingBurst;`
	`  completed++;`
	`  continue;`
	`}`
		Non-Preemptive Branch: If preemption is disabled, process is given CPU and executes to completion without interruption.

207–214	`if (currentRunning != highest) {`
	`  if (currentRunning != null) {`
	`  addGanttBlock(ganttChart, currentRunning.original, blockStartTime, currentTime);`
	`  }`
	`  currentRunning = highest;`
	`  blockStartTime = currentTime;`
	`}`
		Context Switch Detection: If a different process has higher priority than the currently running one, the current process yields, its executed slice is added to Gantt, and `highest` takes the CPU.

217–228	`int step = currentRunning.remainingBurst;`
	`for (TaskState s : states) {`
	`  if (s.remainingBurst > 0 && s.original.getArrivalTime() > currentTime) {`
	`  if (s.original.getPriority() < currentRunning.original.getPriority()) {`
	`  int timeUntilArrival = s.original.getArrivalTime() - currentTime;`
	`  if (timeUntilArrival > 0 && timeUntilArrival < step) {`
	`  step = timeUntilArrival;`
	`  }`
	`  }`
	`  }`
	`}`
		Event-Driven Preemption Step (Critical Algorithm): Rather than incrementing clock 1 by 1 (which is slow), it calculates `step`: the exact time until the process finishes OR until a higher-priority patient arrives in the future. If a higher priority patient arrives in 10 min and remaining burst is 30 min, `step = 10`.

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
231–232	`currentTime += step;` `currentRunning.remainingBurst -= step;`	Advances clock by `step` and decrements process's remaining burst time.

234–240	`if (currentRunning.remainingBurst == 0) {`
	`  currentRunning.completionTime = currentTime;`
	`  completed++;`
	`  addGanttBlock(ganttChart, currentRunning.original, blockStartTime, currentTime);`
	`  currentRunning = null;`
	`}`
		If process has 0 remaining burst, marks `completionTime`, increments `completed` count, records Gantt block, and releases CPU.

243–255	`for (TaskState s : states) {`
	`  int tat = s.completionTime - s.original.getArrivalTime();`
	`  int wt = tat - s.original.getBurstTime();`
	`  totalTurnaroundTime += tat;`
	`  totalWaitingTime += wt;`
	`}`
		Loops over all finished processes to calculate final TAT, WT, and averages.

Section 4: Round Robin (RR) with Time Quantum (Lines 267 – 403)

Preemptive Time Slicing Fair physician ward rounds.

```
RR (q=20) Gantt: [0--T1--20] [20--T2--40] [40--T3 (Done)--60] [60--T1--80] [80--T4--100] [100--T2 (Done)--110]
[110--T1 (Done)--115] [115--T4--135] [135--T4 (Done)--145]
Avg WT: 58.75 min | Avg TAT: 95.00 min
```

Line(s)	Java Source Code	Detailed Line-by-Line Explanation

282–285	`public ScheduleResponseDto scheduleRoundRobin(List<ProcessTaskDto> tasks, int timeQuantum) {`
	`  if (timeQuantum <= 0) {`
	`  timeQuantum = 20;`
	`  }`
		Defensive check: If invalid non-positive quantum is passed, falls back to standard 20-minute time slice.

290–301	`class RRTaskState {`
	`  final ProcessTaskDto original;`
	`  int remainingBurst;`
	`  int completionTime;`
	`  boolean inQueue;`
	`  ...`
	`}`
		Round Robin state wrapper. The `boolean inQueue` flag is crucial: it prevents the same process from being added to the ready queue multiple times simultaneously.

309–311	`states.sort(Comparator.comparingInt(s -> s.original.getArrivalTime()));` `Queue<RRTaskState> readyQueue = new LinkedList<>();`	Sorts processes by arrival time initially and instantiates the FIFO Queue representing the OS Ready Queue.

317–322	`for (RRTaskState s : states) {`
	`  if (s.original.getArrivalTime() <= currentTime && !s.inQueue) {`
	`  readyQueue.add(s);`
	`  s.inQueue = true;`
	`  }`
	`}`
		Enqueues all processes that are present at initial time $t=0$.

348–350	`RRTaskState current = readyQueue.poll();` `current.inQueue = false;`	De-queues the head process from the ready queue for execution.

352–356	`int slice = Math.min(current.remainingBurst, timeQuantum);`
	`int startTime = currentTime;`
	`int endTime = startTime + slice;`
	`currentTime = endTime;`
	`current.remainingBurst -= slice;`
		Time Quantum Allocation: The process executes for `slice = min(remainingBurst, timeQuantum)`. If remaining burst is 5 min and quantum is 20, it runs for 5 min. Clock advances to `endTime`.

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
358–365	`// CRITICAL OS RULE: Enqueue new arrivals BEFORE re-adding current for (RRTaskState s : states) { if (s != current && s.remainingBurst > 0 && s.original.getArrivalTime() <= currentTime && !s.inQueue) { readyQueue.add(s); s.inQueue = true; } }`	**Crucial OS Rule for Viva:** Any newly arrived processes during this time slice must be enqueued **before** the preempted process is placed back into the queue. This preserves strict FIFO fairness.
367–379	`if (current.remainingBurst > 0) { readyQueue.add(current); current.inQueue = true; ganttChart.add(new GanttBlockDto(..., "Slice", startTime, endTime)); } else { current.completionTime = currentTime; completed++; ganttChart.add(new GanttBlockDto(..., "Completed", startTime, endTime)); }`	If process still has remaining burst, re-enqueues it at the tail and logs a "Slice" block. Otherwise, marks `completionTime`, increments `completed`, and logs a "Completed" block.

Section 5: Shortest Job First (SJF / SRTF) (Lines 405 – 526)

Provably Optimal Average Waiting Time

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
466–470	`SJFState shortest = available.stream() .min(Comparator.comparingInt((SJFState s) -> s.remainingBurst) .thenComparingInt(s -> s.original.getArrivalTime())) .orElse(available.get(0));`	**SJF Selection Rule:** Always selects the process with the smallest remaining burst time. In preemptive mode (SRTF), newly arrived shorter jobs preempt longer ones.

Section 6: Helper Methods & Default Hospital Data (Lines 528 – 585)

Line(s)	Java Source Code	Detailed Line-by-Line Explanation
532–550	`public List<ProcessTaskDto> prepareTasks(List<ProcessTaskDto> tasks) { if (tasks != null && !tasks.isEmpty()) { ... sanitization loop ... return copy; } return getDefaultHospitalTasks(); }`	**Defensive Input Sanitizer:** • Enforces minimum burst time of at least 1 min (`Math.max(1, burst)`). • Enforces non-negative arrival time (`Math.max(0, arrival)`). • If input list is null or empty, falls back to canonical database tasks.
560–566	`public List<ProcessTaskDto> getDefaultHospitalTasks() { return Arrays.asList(new ProcessTaskDto("T1", "John Doe (Angioplasty)", 0, 45, 1), new ProcessTaskDto("T2", "Anita Roy (Ventilation)", 10, 30, 2), new ProcessTaskDto("T3", "Robert Smith (Checkup)", 15, 20, 3), new ProcessTaskDto("T4", "Pooja Verma (Trauma)", 25, 50, 1)); }`	Defines the canonical benchmark dataset matching `oracle_seed.sql`. Enables independent unit testing without an active database connection.
568–580	`private void addGanttBlock(List<GanttBlockDto> chart, ProcessTaskDto task, int start, int end) { if (start >= end) return; if (!chart.isEmpty()) { GanttBlockDto prev = chart.get(chart.size() - 1); if (prev.getProcessId().equals(task.getProcessId()) && prev.getEndTime() == start) { prev.setEndTime(end); return; } } chart.add(new GanttBlockDto(...)); }`	**Intelligent Gantt Block Merging:** In preemptive simulations, a process might execute across multiple consecutive steps without being preempted. This helper merges adjacent blocks of the same process into a single continuous block, preventing fragmented visual charts.
582–584	`private double roundTwoDecimals(double val) { return Math.round(val * 100.0) / 100.0; }`	Utility to format floating-point metric averages to 2 decimal places.

🎓 Final Evaluator Cheat Sheet for SchedulerService.java

Evaluator Question	Winning Academic Answer
"Why is FCFS unsuitable for hospitals?"	It causes the *Convoy Effect*: emergency patients with high urgency wait behind routine long appointments.

Evaluator Question	Winning Academic Answer
"How does your code prevent Round Robin unfairness?"	Lines 358–365 enqueue new arriving tasks *before* re-adding the preempted task to the ready queue.
"What happens if no patient is available?"	Lines 68–76 record an `IDLE` Gantt block and fast-forward the clock to avoid infinite loops or crashes.